



Photo by Pezibear on pixabay

Lecture 8

—

Artificial Neural Networks (ANN)

Kleiner Fuchs (*Aglais urticae*)

- ♦ Der Kleine Fuchs ist ein Schmetterling (Tagfalter) der Familie der Edelfalter, er wird auch häufig „Nesselfalter“ genannt.
- ♦ Orange-braune Flügel mit leuchtend blauen Punkten am Rand: Der Kleine Fuchs ist wegen seiner hübschen Gestalt überaus beliebt.
- ♦ In Deutschland ist der Kleine Fuchs im Sommer häufig anzutreffen und gehört zu den populärsten Schmetterlingsarten überhaupt.
- ♦ Die Raupen des Kleinen Fuchses ernähren sich fast ausschließlich von der Großen Brennnessel (*Urtica dioica*), weshalb man die Falter oft in der Nähe dieser Pflanzen antrifft.
- ♦ Die Falter erreichen eine Flügelspannweite von 40 bis 50 Millimetern.

Quellen:

* https://de.wikipedia.org/wiki/Kleiner_Fuchs

* <https://www.plantura.garden/gruenes-leben/schmetterlinge-die-10-beliebtesten-heimischen-schmetterlingsarten>



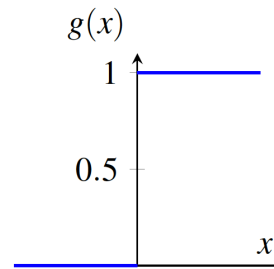
Artificial Neural Networks

1. Activation Functions
2. Backpropagation
3. Training a Neural Network
4. Loss Functions and Network Setup
5. Code Examples

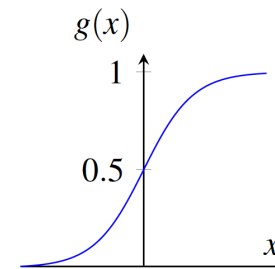


Overview

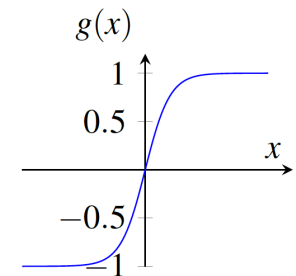
- ◆ Chosen to have properties to either enhance or simplify the network containing the neuron
- ◆ Has to be non-linear in order to implement non-linearly-separable functions by combining multiple neurons
- ◆ Most widely used activation functions



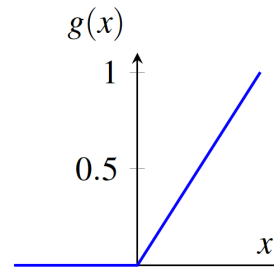
(a) Sprungfunktion



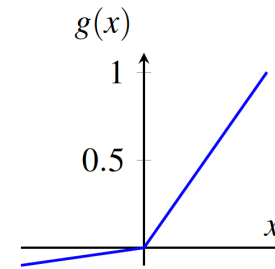
(b) Sigmoid



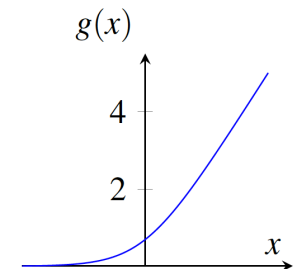
(c) tanh



(d) ReLU



(e) Leaky ReLU



(f) Softplus

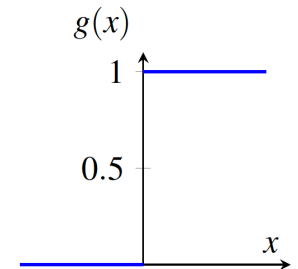


Step Function and Identity Function

♦ Step Function

- Binary output: depending on whether the input meets a specified threshold θ .

$$g(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$



- Division of the space of inputs by a hyperplane.
- Specially useful in the last layer of a network intended to perform binary classification
- Can be approximated from other sigmoidal functions by assigning large values to the weights.
- Big drawback: non-continuous and therefore non-differentiable at the threshold value.

♦ Identity Function

- Simple output the sum without using an activation function

$$g(x) = x$$

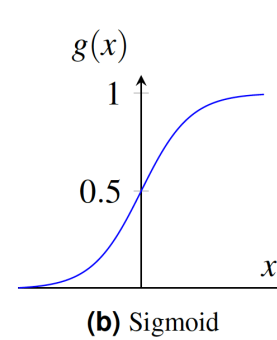
- Widely used in the final layer of network used for regression problems.



Sigmoid and Tangens Hyperbolicus

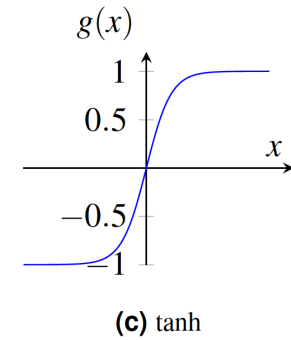
- ◆ Sigmoid Function $g_s(x)$

$$g_s(x) = \frac{1}{1+e^{-x}}$$



- ◆ Tangens Hyperbolicus $g_t(x)$

$$g_s(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



- ◆ First derivate can be easily calculated for both:

$$g'_s(x) = g_s(x) (1 - g_s(x))$$

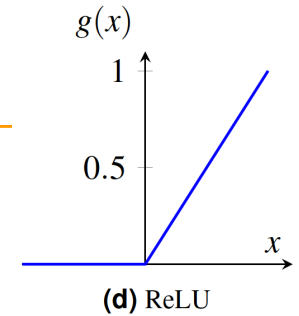
$$g'_t(x) = 1 - g_t^2(x)$$



ReLU, leaky ReLU and Softplus

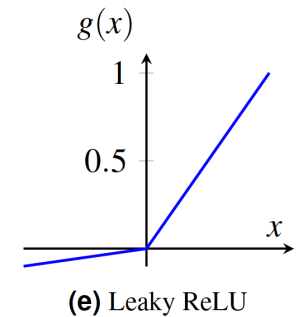
- ◆ ReLU

$$g_r(x) = \max(0, x) = \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} \quad g'_r(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0 \end{cases}$$



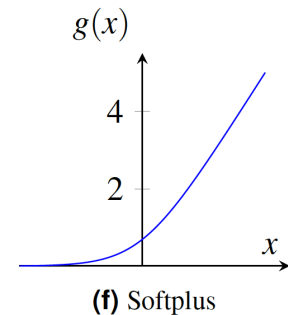
- ◆ Leaky ReLU

$$g_l(x) = \begin{cases} 0.01x & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} \quad g'_l(x) = \begin{cases} 0.01 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0 \end{cases}$$



- ◆ Softplus

$$g_p(x) = \ln(1 + e^x) \quad g'_p(x) = \frac{1}{1 + e^{-x}}$$



- ◆ Currently most widely used activations functions
- ◆ ReLU is continuous and almost differentiable (chose 0 or 1 at the threshold). Softplus is a smooth approximation of ReLU and differentiable.



Softmax

- ◆ Cannot be shown graphically
 - Output does depend on all neurons in the layer, not on a single neuron x as for the other functions.
 - Let us collect the outputs x_i of all these neurons in a vector $\vec{x}$ and
Let $\delta_{ij} = 1$ if $i = j$ and 0 otherwise.

- ◆ Softmax
$$g_{mj}(\vec{x}) = \frac{e^{x_j}}{\sum_i e^{x_i}} \quad \frac{\partial g_{mj}(\vec{x})}{\partial x_i} = g_{mj}(\vec{x})(\delta_{ij} - g_{mi}(\vec{x}))$$

- ◆ Smooth and differentiable approximation of the maximum function
 - Increases large values and decreases small values.
 - Main advantage: sum of all values is always equal to 1 → values can be interpreted as probabilities.
- ◆ Often used in the final layer of a network used for classification problems.



Exercises

Exercise 1

Activation Functions





Artificial Neural Networks

1. Activation Functions
2. Backpropagation
3. Training a Neural Network
4. Loss Functions and Network Setup
5. Code Examples



Video

Video

Backpropagation

<https://www.youtube.com/watch?v=llg3gGewQ5U>





Exercises

Exercise 2

Backpropagation





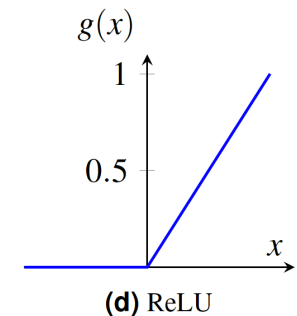
Artificial Neural Networks

1. Activation Functions
2. Backpropagation
3. Training a Neural Network
4. Loss Functions and Network Setup
5. Code Examples



Vanishing Gradient Problem

- ♦ **Vanishing Gradient Problem:** first derivative (gradient) of Sigmoid and tanh is large around the threshold (0), but becomes closer and closer to 0 the further away (as the functions become flatter and flatter) → the gradient vanishes.
- ♦ Sigmoid and tanh almost never used in practise any more.
- ♦ (Leaky) ReLU and SoftPlus are currently most widely used activations functions.





Optimizations regarding the training algorithm

- ◆ **SGD optimizer** (stochastic gradient descent) introduced in the videos → very simple!
- ◆ More advanced optimizer algorithms with two main ideas:
 - **Adapting the learning rate during training:**
 - SGD takes a step of a certain fixed size in the direction of the negative gradient.
 - Observation: good to make large steps at the beginning of the training (when far away from the local minimum). Good to make smaller steps, when closer to the minimum, (in order to not "overshoot")
→ adapt the learning rate dynamically during training.
 - **Moments:**
 - do not only use the current gradient as the direction of descent
→ also factor in the gradients of the previous step with exponentially decreasing strength.
- ◆ **RMSprop optimizer**
 - Combines both ideas
 - Very popular



Batches and Epochs

- ◆ **MiniBatches**: known from the video
- ◆ Typically: divide training examples into three sets
 - **test set**: to be used for evaluation after training (about 10-15% of the data)
 - **training set**: used for actual training (about 70-80% of the data)
 - **validation set**: for validation tuning during the training (about 10-15% of the data)
- ◆ **Resulting process**:
 - 1) divide the **training set** into the batches (e.g. 128 training examples per batch)
 - 2) train the network on one batch
 - 3) once all batches have been used for training (which is called an **epoch**), compute loss and some quality metric (e.g. accuracy) based on the **validation set**
 - 4) stop the training if either the maximal number of epochs has been reached or the quality metric has reached a given threshold, otherwise repeat from step 2
- ◆ Compute quality metric (and maybe loss) on the **test set**



Artificial Neural Networks

1. Activation Functions
2. Backpropagation
3. Training a Neural Network
4. Loss Functions and Network Setup
5. Code Examples



Loss Function

- ♦ **Cost function / Loss function / Objective function**: to be optimized by the network
 - **(mean) squared error**: introduced in the video
 - **cross-entropy / log loss**: measures the performance of a classification model whose output is a probability value between 0 and 1.

$$\epsilon(x) = -\sum_j y_j(x) \ln(d_j(x)) \text{ where}$$

x is a training example,

$d_j(x)$ is the output of neuron j in the output layer

$y_j(x)$ if the desired output (0 for all but the one neuron encoding the correct class (for which is it 1)).

- Definition derived from the definition of entropy in information theory.
 - Cross-entropy loss increases as the predicted probability diverges from the actual label. So predicting a probability of .017 when the actual observation label is 1 would be bad and result in a high loss value.
 - Perfect model: log loss = 0.
- Special case: binary classification
 - only one output neuron → use **binary cross-entropy**: $\epsilon(x) = -y(x) \ln(d(x)) + (1 - y(x)) \ln(1 - d(x))$



Popular neural network setups for different problems

- ◆ Depending on the problem at hand, different choices for the activation function of the output layer and the loss function can be recommended:
 - **binary classification**
 - output layer: only one neuron, which is supposed to be 0 or 1.
 - Sigmoid as activation function and binary cross-entropy as loss function often work quite well.
 - **multiple disjoint classes**
 - output layer: one neuron per class. Exactly one of these is supposed to be 1, all others should be 0.
 - Softmax as activation function and cross-entropy as loss function often work quite well.
 - **multiple non-disjoint classes**
 - output layer: one neuron per class, but now multiple neurons can and should be 1.
 - Sigmoid as activation function and the sum of binary cross-entropy as loss function (each neuron can be considered a binary classification problem) often work well.
 - **Regression**
 - output layer: one neuron. instead of predicting a class label, the network has to compute a (numeric) value.
 - Identity function as activation function and the squared error as loss function often work well.
- ◆ Activation Function for Hidden layers: ReLU or one of its variants (leakyReLU, Softplus).



Artificial Neural Networks

1. Activation Functions
2. Backpropagation
3. Training a Neural Network
4. Loss Functions and Network Setup
5. Code Examples



Exercise 3

Demo: ANN in Action
Exercise: ANN Optimization





Key Takeaways

- ◆ Structure of an ANN
- ◆ Feed Forward and Backpropagation
 - SGD (stochastic gradient descent)
 - RMSprop optimizer
 - MiniBatches and Epochs
- ◆ Activation Function and Loss Function
 - Sigmoid, ReLU, ...
 - Mean squared error, cross entropy/log loss
- ◆ Vanishing Gradient Problem
- ◆ Here: Basic ANN – more advanced architectures see class on Deep Learning

